A new implementation of Short-paths

Paul-Elliot Anglès d'Auriac (Tarides) Ulysse Gérard (Tarides)
Leo White (Jane Street)

Introduction

What is short-paths?

The OCaml type system, and its module system in particular, is eminently malleable. One of the consequences of this flexibility is that the same entities can often be referred to by multiple paths in the same environment.

For example, if you consider the following snippet:

```
module X = struct
   type t = Unix.error
end

module Y = X

open X;;

let err = Unix.EMLINK;;
```

The type of `err`, at the location of its definition, could be referred to as `Unix.error`, `X.t`, `Y.t`, or `t`. And this set of candidates can grow quickly in large libraries, especially when they rely heavily on features such as `include`. The conventional encapsulation of libraries with wrapper modules, named with double underscores `__`, is also a common source of alternative paths that we should avoid showing to the user.

The mechanism choosing the best type is what we call "short-paths". It is involved each time a type is printed; this happens for example when the compiler prints an error, or when Merlin/OCaml-LSP prints the type of an expression as a result of a user query.

It's not obvious how to define "best path", but the cost function should at least: 1. consider the number of components (separated by dots), prioritizing paths with fewer components, and 2. depreciate components containing a double underscore.

(Many other factors could be taken into account to break ties: favor candidates appearing close in the buffer, favor predefined types, etc.)

Existing implementations

Short-paths is a feature that is surprisingly tricky to get right, that is, to provide a reasonable answer in a reasonable amount of time. In fact there already exist two implementations in the ecosystem.

One lives in the compiler itself and is used when printing error messages with the option

`-short-paths` (enabled by default by Dune while in dev mode). It performs a lazy breadth-first search in the environment, one level at a time, until it finds an adequate candidate. It can miss good candidates by stopping too early and is rather costly. This is not a problem for compiler output, where only a few types are printed in an error, but it is not acceptable for real-time applications such as Merlin.

For these reasons, there is a different implementation of short-paths in Merlin. It is an extremely complex machine in comparison with the compiler one, but it is able to explore deeper in the environment, faster, by smartly cutting branches when possible. It provides better results, faster, but still misses some cases, and its complexity makes it quite hard to maintain. The plan was to upstream it to the compiler but it never happened.

Both have their flaws. This led co-author Leo White, the creator of the current Merlin implementation, to propose a new design, expected to be simpler, faster and more accurate.

Remark on terminology: in the compiler the term "path" is an internal notion that designates a valid path in a given typing environment. Names, or "paths" written by the users in the sources are called "longidents" in the typer. In this presentation we liberally use the word "path" to name what in the compiler codebase is actually a "longident".

The new design

We introduce a notion of domain of discourse $\mathcal{D}$ which is the set of paths that should be considered when shortening. One fundamental difference in the new design is that some preprocessing is done during typing and saved as part of the type signature in the `cmi` files: each (module, type, value, etc) declaration is enriched with the set of paths that should be added to the discourse if it is *used*.

We use a two-step process to build $\mathcal{D}$: we first gather $\mathcal{U}$, the set of paths explicitly used in the source file. Then we apply a number of rules over the paths in $\mathcal{U}$ to build the complete domain of discourse $\mathcal{D}$ for the module. $\mathcal{U}$ is accumulated during typing, but $\mathcal{U} \to \mathcal{D}$ is performed only before printing so as not to slow down compilation.

Once we have $\mathcal{D}$ we will process the paths in it in increasing cost order until we find an appropriate candidate.

An overview of the rules

We only describe a subset of the rules here, in the hope to give the reader a feeling of the whole process.

Rules that build $\mathcal{U}$, the set of used paths

- Rule $\mathcal{U}_1$: Paths defined in the file are in $\mathcal{U}$ (eg. `x` in `let x = ...`)
- $\mathcal{U}_2$: Paths used in the file are in $\mathcal{U}$ (eg. `List.map` in `let _ = List.map`)
- $\mathcal{U}_3$: Paths defined via an `open` or `include` are in $\mathcal{U}$ (eg. `include List` brings `fold_left`)

Rules that build the discourse $\mathcal{D}$

- Rule $\mathcal{D}_2$: If a path is in $\mathcal{U}$, it is in $\mathcal{D}$.
- $\mathcal{D}_3$: If a module path is in $\mathcal{U}$ then all the paths of its subcomponents are in $\mathcal{D}$.(eg. `List` $\in \mathcal{U} \Rightarrow$ `List.t` $\in \mathcal{D}$)

- $\mathcal{D}_4$: If a value path is in $\mathcal{U}$ and its value description was written by a user – as opposed to being inferred – then the paths used in that description are in $\mathcal{D}$. (eg. if `val x : Y.t`, then $\texttt{x} \in \mathcal{U} \Rightarrow \texttt{Y.t} \in \mathcal{D}$)
- $\mathcal{D}_6$: If a type path is in $\mathcal{U}$ then any paths used in its equation or representation are in $\mathcal{D}$.
- $\mathcal{D}_{12}$: If a module path `m` in $\mathcal{D}$ - note $\mathcal{D}$ not $\mathcal{U}$ - is a module alias with target `n` and another path `p` in $\mathcal{D}$ includes `n` within it, then the path obtained by substituting the `m` for `n` in `p` is also in $\mathcal{D}$. (eg. if `module M = X.Y`, then $\texttt{X.Y.t} \in \mathcal{U} \Rightarrow \texttt{M.t} \in \mathcal{D}$) This rule is applied lazily: during typing we accumulate a set of substitutions along with $\mathcal{U}$ which is used when building $\mathcal{D}$ before printing.

The shortening algorithm

Even if this used-first way of filling $\mathcal{D}$ should result in a smaller search space than the trivial implementations, some care must be taken to actually compute the shortest path in the most economical way possible. Here is how we proceed:

1. When asked to shorten a path P we apply to it the substitutions we collected as part of rule $\mathcal{D}_{12}$ and add all these paths to $\mathcal{D}$.
2. We place all the paths of $\mathcal{D}$ in a priority-list sorted by their cost (length + double underscore malus)
3. For every path in the first level of the list (which contains the shortest names) we check if it is a valid name in the current environment. If it is we canonicalize that path and add it to a table T mapping canonicalized paths to a sorted list of names.
4. If the table T contains an entry for the canonical form of P that is valid in the current environment we have our answer. If not we loop back to step 3.

Several optimisations are made in the current implementation to improve performance when successively shortening paths in the same buffer or when printing large module signatures (our test-suite on base prints signatures of more than 14M characters). Some of these tricks involve reusing the table T and only retrying invalid entries in the priority list when it makes sense.

Results

We are currently working on a prototype implementation based on oxcaml. We get much better results when randomly printing types from `Base` compared to the current implementation in Merlin, with almost all occurrences of double-underscored paths avoided.

Performances are currently slightly worse than the existing implementation in Merlin (which itself is quite fast), but we expect to be able to have a faster algorithm in the end.

Because this new algorithm requires storing discourses in the AST of types, changes to the upstream compiler will be required if we want to bring these improvements to every OCaml programmer.